JORDAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Computer and Information Technology

Department of Computer Science

ScoRev: A Hybrid Framework for Multi-Dimensional Software Quality Assessment through Static Analysis and Knowledge-Distilled Language Models

Final Project Report Submitted to
The Department of Computer Science

In Partial Fulfillment of the Requirements for the Degree of
Bachelors of Science in Computer Science

Prepared by:

Sara Jaradat [165199]
Ruba AL Harahshah [158415]
Huda Shqerat [163547]
Hala ALshouha [147651]

Supervisor:
Yaser Jararweh

June 2026

نموذج حقوق الملكية الفكرية لمشاريع التخرج في قسم علوم الحاسوب

يتم قراءة وتوقيع هذا النموذج من قبل الطلاب المسجلين لمشاريع التخرج في قسم علوم الحاسوب

تعود حقوق الملكية الفكرية لمشاريع التخرج ونتائجها (مثل براءات الاختراع أو أي منتج قابل للتسويق) إلى جامعة العلوم والتكنولوجيا الأردنية، وتخضع هذه الحقوق إلى قوانين وأنظمة و تعليمات الجامعة المتعلقة بالملكية الفكرية وبراءات الاختراع.

بناءا على ما سبق أوافق على ما يلي:

1) أن أحفظ كافة حقوق الملكية الفكرية لجامعة العلوم والتكنولوجيا الأردنية في مشروع التخرج.
2) أن ألتزم بوضع اسم جامعة العلوم والتكنولوجيا الأردنية و أسماء جميع الباحثين المشاركين في المشروع على أي نشرة علمية للمشروع كاملا أو لنتائجه. و يشمل ذلك النشر في المجلات و المؤتمرات العلمية عامة أو النشر على المواقع الإلكترونية أو براءات الاختراع أو المسابقات العلمية.
3) أن ألتزم بأسس حقوق التأليف المعتمدة في جامعة العلوم والتكنولوجيا الأردنية.
4) أن أقوم بإعلام الجهة المختصة في الجامعة عن أي اختراع أو اكتشاف قد ينتج عن هذا المشروع و أن ألتزم السرية التامة في ذلك و أن أعمل من خلال الجامعة على الحصول على براءة الاختراع التي قد تنتج عن هذا المشروع.
5) أن تكون جامعة العلوم والتكنولوجيا الأردنية هي المالك لأي براءة اختراع قد تنتج عن هذا المشروع و تشمل هذه الملكية حق الجامعة في إعطاء التراخيص و التسويق و البيع كمؤسسة راعية و داعمة لكافة الأنشطة البحثية. ويكون حق للطالب شمول اسمه على براءة الاختراع كأحد المخترعين، و في حال تم إعطاء تراخيص أو تسويق و بيع لأي من منتجات المشروع يمنح المخترعون بما فيهم الطالب نسبة من الإيرادات حسب تعليمات البحث العلمي في جامعة العلوم والتكنولوجيا الأردنية.

إسم الطالب	التوقيع
إسم الطالب	التوقيع
إسم الطالب	التوقيع
إسم المشرف	**التوقيع**

تاريخ .....

ScoRev: A Hybrid Framework for Multi-Dimensional Software Quality Assessment through Static Analysis and Knowledge-Distilled Language Models

Ruba AL Harahshah*, Sara Jaradat*, Hala ALshouha*, Huda Shqerat*, Yaser Jararweh*
*Computer Science Dept., Jordan University of Science and Technology, Irbid, Jordan
{ rwalharahshah22,szjaradat22,haalshouha23,heshqerat22}@cit.just.edu.jo, yijararweh@just.edu.jo

1. ABSTRACT

***Abstract*—Software quality assessment remains a significant challenge as software systems grow in complexity and AI-generated code becomes increasingly common. Traditional static analysis tools provide deterministic and explainable results but lack semantic understanding, while Large Language Models (LLMs) offer contextual reasoning yet suffer from inconsistency and limited reproducibility.**

This paper presents ScoRev, a hybrid software quality assessment framework that combines deterministic static analysis with a knowledge-distilled language model. The framework evaluates source code across three key dimensions: security, complexity, and maintainability. A deterministic engine generates quantitative quality scores and line-level findings, while the distilled model provides semantic insights, identifies complementary issues, and produces actionable recommendations.

To support this approach, a large-scale dataset was constructed from open-source Python repositories through a grounded knowledge distillation process that integrates source code, static-analysis outputs, and semantic reviews. Experimental results show that integrating deterministic engine findings with the distilled language model improves software-quality issue detection, achieving a 43% relative F1 improvement over the model-only baseline while providing explainable and actionable review recommendations. ScoRev provides a practical foundation for automated code review and software quality assurance for both human-written and AI-generated software.

***Index Terms*—Software Quality Assessment, Static Analysis, Large Language Models, Knowledge Distillation, Code Review, Software Engineering**

2. Project Goals and Objectives

The primary goal of this research is to develop ScoRev, a multi-dimensional software quality assessment and improvement framework that combines deterministic, metric-driven analysis with semantic reasoning capabilities derived from Large Language Models (LLMs). The framework is designed to provide explainable, reproducible, and actionable assessments of source code quality across the security, complexity, and maintainability dimensions while supporting iterative code-quality improvement.

To achieve this goal, the following objectives are pursued:

- Develop a deterministic software quality assessment engine capable of producing structured issue-level findings and quantitative quality scores across the security, complexity, and maintainability dimensions.
- Design an explainable scoring methodology grounded in established software engineering metrics and mathematically defined aggregation models, ensuring that all quality assessments remain transparent, reproducible, and defensible.
- Construct a large-scale distillation dataset from diverse open-source Python repositories, linking source code, static-analysis findings, quality scores, and expert-level semantic review outputs.
- Fine-tune a compact language model to complement deterministic analysis by generating semantic insights, identifying higher-level quality concerns, producing actionable recommendations, and suggesting code improvements beyond the capabilities of rule-based analysis.
- Investigate the relationship between metric-driven quality assessment and semantic code understanding by comparing deterministic findings with LLM-generated assessments and observations.
- Establish a reproducible research framework that supports future work in software quality assessment, automated code review, AI-assisted software engineering, and quality assurance for AI-generated code.

3. Introduction

Software quality assessment has long been a major challenge in software engineering. With the increase in complexity and scale of modern software systems and the development of new systems, keeping source code secure, maintainable and computationally efficient is essential in order to avoid technical debt and reduce operational risk and to maintain long-term evolution [1], [2]. Traditionally, software quality assessment has been done using static analysis tools, software metrics, code reviews, and expert judgment [3]. These techniques have been very successful, but rapidly evolving AI-assisted software development has made the problem much more complex. As code generation becomes increasingly automated, the bottleneck shifts from producing code to systematically evaluating and improving its quality before deployment. The recent

large language models (LLMs) can generate large amounts of functional code from natural language prompts, enabling developers to create complete software components, applications, and workflows with unprecedented speed [4].Systems such as GitHub Copilot, CodeLlama, and other code-generation assistants have shifted software development toward an increasingly AI-driven paradigm [5]. However, code generation does not guarantee code quality. Empirical studies have repeatedly shown that AI-generated code may contain security vulnerabilities, inefficient implementations, maintainability deficiencies, architectural weaknesses, and hidden logic flaws despite appearing syntactically correct and functionally valid [6], [7].

Existing automated assessment approaches provide only partial solutions. Static analysis tools offer reproducibility, transparency, and traceability by detecting vulnerabilities and structural defects through predefined rules, software metrics, and pattern-based reasoning [3], [1]. Their outputs are deterministic, explainable, and suitable for objective quality measurement. However, static analysis remains fundamentally constrained by its reliance on syntactic and structural evidence. It cannot reliably reason about software intent, contextual design choices, architectural trade-offs, or semantic consequences that emerge only through higher-level understanding of code behavior [8].

Large language models provide a complementary capability. By leveraging semantic reasoning, LLMs can interpret program behavior, identify latent design concerns, explain quality deficiencies, and generate human-readable recommendations [9]. Recent work has demonstrated promising results in code review generation, defect detection, vulnerability analysis, and program comprehension [10], [11]. Nevertheless, LLM-based assessments introduce challenges of their own, including non-determinism, limited reproducibility, inconsistent judgments, and occasional hallucinations that can undermine trust when quality assessments are expected to support engineering decisions [12], [13].

Beyond these limitations lies a deeper challenge. Most existing approaches evaluate software quality through isolated metrics, issue counts, or task-specific review outputs. Such assessments rarely provide a unified and interpretable representation of software quality across multiple dimensions. In practice, software quality is inherently multidimensional: a system may be secure yet difficult to maintain, maintainable yet computationally inefficient, or efficient yet vulnerable to security risks [14]. Effective assessment therefore requires mechanisms capable of simultaneously quantifying multiple quality dimensions while also capturing semantic properties that cannot be reduced to static metrics alone.

This paper presents **ScoRev**, a multidimensional software quality assessment and improvement framework designed to address this challenge. ScoRev combines a deterministic quality measurement engine with a distilled semantic review model to provide both quantitative and qualitative perspectives on software quality. Rather than treating code review as a purely generative task, the framework models software quality across three fundamental dimensions: *Security*, *Complexity*, and *Maintainability*. The deterministic engine transforms source code into calibrated quality scores and structured line-level findings using literature-grounded metrics, mathematically defined scoring formulations, and explicitly justified design decisions. Every scoring parameter is traceable to either established software-engineering literature, a documented modeling rationale, or a transparent calibration process, enabling explainable and auditable quality assessment.

To complement this quantitative foundation, ScoRev incorporates a compact language model trained through grounded knowledge distillation. A large-scale software-quality dataset is constructed from diverse open-source repositories and enriched with deterministic quality measurements, structured issue reports, semantic assessments, recommendations, and corrective guidance. The resulting model extends deterministic analysis by generating semantic issues, contextual insights, actionable recommendations, and code-improvement suggestions that would otherwise remain inaccessible to purely metric-driven approaches.

The framework is further motivated by an increasingly important deployment scenario: quality assurance for AI-generated software. In this setting, ScoRev functions as a post-generation assessment layer that enables iterative quality improvement. Generated code is evaluated, weaknesses are identified across multiple quality dimensions, corrective recommendations are produced, and revised code can subsequently be re-assessed using the same framework. This establishes a feedback-driven quality improvement loop that transforms software quality assessment from a passive evaluation activity into an active quality-enhancement process.

The contributions of this work are summarized as follows:

- We propose **ScoRev**, a multidimensional software quality assessment and improvement framework that combines deterministic quality measurement with semantic reasoning.
- We introduce a transparent scoring methodology for evaluating software quality across Security, Complexity, and Maintainability dimensions using literature-grounded metrics and mathematically defined scoring formulations.
- We construct a large-scale software-quality dataset that integrates source code, deterministic quality assessments, structured findings, semantic reviews, recommendations, and corrective guidance for knowledge distillation.
- We develop a distilled semantic review model capable of generating semantic issues, contextual insights, actionable recommendations, and code-improvement suggestions that complement deterministic analysis.
- We demonstrate a practical quality-assurance workflow for AI-generated software in which assessment, recommendation, remediation, and re-assessment form an iterative quality improvement cycle.

4. Related Work

A. Static Analysis for Software Quality Assessment

Static analysis remains one of the most widely adopted approaches for automated software quality assessment. These

techniques analyze source code without execution and identify potential vulnerabilities, coding standard violations, maintainability concerns, and structural complexity issues. Industrial tools such as SonarQube and Bandit are widely used due to their deterministic behavior and reproducibility. However, several studies have highlighted the limitations of static analysis systems, particularly their dependence on handcrafted rules and their tendency to generate false positives and context-insensitive warnings [15].

Lenarduzzi et al. [15] conducted a large-scale comparison of six widely used static analysis tools and reported substantial disagreement among analyzers regarding issue detection and severity assessment. More recent studies have explored augmenting static analysis with LLM-based reasoning. Abtahi and Azim [16] demonstrated that combining static analysis outputs with LLMs can improve automated code quality recommendations. Similarly, LLM4Code [17] showed that chain-of-thought reasoning can reduce false positives generated by traditional vulnerability analyzers. While these approaches enhance the usefulness of static analysis, they still rely on external reasoning mechanisms and do not provide comprehensive multi-dimensional quality assessment.

B. LLM-Based Code Review and Quality Evaluation

Recent advances in Large Language Models (LLMs) have transformed software engineering research. Models such as CodeReviewer introduced specialized pre-training objectives for automated code review and demonstrated the ability to generate meaningful developer feedback from code changes [18]. Subsequent approaches expanded LLM applications to code understanding, vulnerability detection, defect localization, and software quality evaluation.

HuCoSC (2024) proposed an LLM-based framework for human-like code quality evaluation and reported strong correlation between model-generated scores and human judgments [19]. Similarly, RACE (2024) introduced a multi-dimensional benchmark that evaluates readability, maintainability, correctness, and efficiency of generated code [20]. Despite their promising performance, these approaches rely primarily on model-generated assessments and therefore inherit the non-deterministic nature of LLM reasoning. Furthermore, Raina et al. [21] demonstrated that LLM-based evaluators are vulnerable to adversarial prompting, while Ullah et al. [22] showed that even state-of-the-art LLMs struggle to reliably identify software vulnerabilities.

C. Hybrid Static Analysis and LLM-Based Systems

To overcome the limitations of both static analysis and standalone LLM evaluation, recent research has increasingly focused on hybrid approaches. IRIS [23] combines CodeQL-based static analysis with LLM reasoning to improve vulnerability detection in large software repositories. Similarly, Jaoua et al. [24] proposed multiple integration strategies, including Retrieval-Augmented Generation (RAG), Data-Augmented Training (DAT), and Naïve Output Concatenation (NCO), for combining static analyzers with LLM-based code review generation.

Other studies explored using static analysis outputs as feedback signals for iterative code improvement and vulnerability mitigation. Although these approaches improve review quality and vulnerability detection accuracy, they primarily focus on security-related tasks and do not provide unified frameworks for evaluating multiple software quality dimensions simultaneously. Furthermore, existing systems rarely incorporate explicit validation mechanisms for identifying false-positive and false-negative analyzer outputs.

D. Software Engineering Datasets and Evaluation Frameworks

The increasing adoption of LLMs has led to the development of numerous software engineering datasets and evaluation benchmarks. CodeReviewer [18] introduced a large-scale dataset for automated code review generation, while RACE [20] established a benchmark for evaluating generated code across multiple quality dimensions. More recent studies have explored datasets for vulnerability detection, bug fixing, and code quality evaluation [19], [22].

Despite these efforts, existing datasets typically focus on a single task or quality dimension. Security-oriented datasets emphasize vulnerability detection, whereas code review datasets concentrate on generating textual feedback. Few datasets combine deterministic quality measurements, semantic validation, and multi-dimensional software quality assessment within a unified framework. Moreover, current datasets rarely provide explicit annotations for false-positive and false-negative analysis or consistency validation between static analysis tools and LLM-generated assessments.

E. Research Gap

The literature reveals three major limitations. First, traditional static analysis tools provide deterministic assessments but lack contextual reasoning and semantic explanations [15]. Second, LLM-based quality evaluators offer rich contextual understanding but suffer from non-determinism, hallucinations, and susceptibility to adversarial manipulation [21], [22]. Third, existing hybrid approaches are largely restricted to vulnerability detection or code review generation and do not support comprehensive software quality assessment across multiple dimensions [23], [24].

To the best of our knowledge, no prior work combines deterministic static analysis, multi-dimensional quality scoring, LLM-based semantic validation, false-positive and false-negative identification, and dataset construction within a single reproducible framework. This gap motivates the development of **ScoRev**, which integrates these capabilities to produce both reliable software quality assessments and a large-scale instruction-tuning dataset.

Table 1
SUMMARY OF RELATED WORK

Paper	Methodology	Main Results	Limitations
CodeReviewer (2022)	LLM pre-training for automated code review	Generates high-quality review comments and feedback	Focuses on review generation rather than software quality assessment
Lenarduzzi et al. (2023)	Comparative analysis of static analysis tools	Identified significant disagreement among analyzers	Limited semantic understanding and high false-positive rates
HuCoSC (2024)	LLM-based code quality evaluation	Strong correlation with human judgments	Non-deterministic scoring and limited validation mechanisms
RACE (2024)	Multi-dimensional evaluation benchmark	Evaluates readability, maintainability, and efficiency	Designed for benchmarking rather than assessment pipelines
Raina et al. (2024)	LLM-as-a-Judge evaluation	Exposed vulnerabilities to adversarial prompting	Focused on evaluator robustness rather than code quality
Ullah et al. (2024)	Security vulnerability assessment using LLMs	Analyzed LLM capability in vulnerability detection	Demonstrated inconsistent vulnerability detection
IRIS (2025)	CodeQL + LLM hybrid analysis	Improved vulnerability detection accuracy	Focused primarily on security analysis
Jaoua et al. (2025)	Static analyzers integrated with LLM review generation	Enhanced code review quality using hybrid strategies	Did not provide multi-dimensional quality scoring
LLM4Code (2025)	Chain-of-thought assisted static analysis	Reduced false positives in vulnerability detection	Focused mainly on security findings
Abtahi & Azim (2025)	Static analysis augmented with LLM reasoning	Improved automated code quality recommendations	Lacked dataset construction and validation framework

5. METHODOLOGY

We approach ScoRev as a design-science artifact: a system built to address a concrete engineering problem and evaluated in the context for which it is intended. The problem is a structural limitation shared by the two dominant families of automated code-quality tools. Rule-based static analyzers are precise and fully reproducible, but their reasoning is confined to syntactic and pattern-level properties; they cannot interpret the intent or semantics of code, and they are known to emit findings that do not correspond to genuine quality defects. Learning-based models capture semantic and contextual nuance, but they are non-deterministic, opaque, and prone to unsupported or fabricated judgments—a particular liability when the output is a numerical quality score. ScoRev is designed so that each family supplies what the other lacks: a deterministic, literature-grounded static engine provides a reproducible quantitative foundation, and a compact language model, trained by grounded knowledge distillation, contributes the semantic layer that static analysis cannot reach. This section describes the artifact in the order in which it processes code. Section 5.F presents the architecture and the design principles that govern it. Section 5.G details the static analysis engine and its three scoring dimensions. Section 5.H describes the grounded distillation procedure used to construct the training corpus. Section 5.I specifies the fine-tuning of the student model. Throughout, we treat the engine as an *internal, reproducible reference* rather than an absolute ground truth: it provides a consistent and auditable basis for assessment, not a claim of perfect correctness.

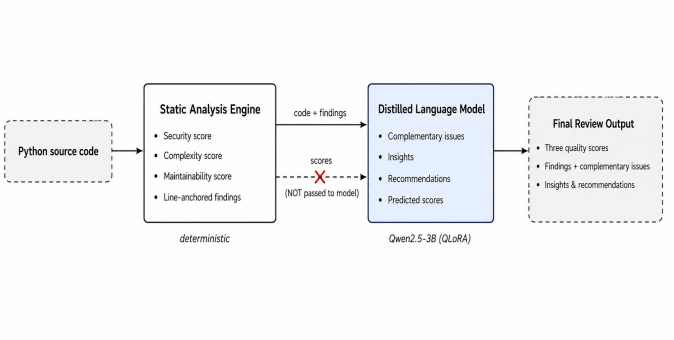

Figure 1. Overview of ScoRev. At inference, source code is processed by the deterministic static analysis engine, which produces three calibrated quality scores and a structured, line-anchored list of findings. The code and the engine's findings—but not its scores—condition the distilled language model, which produces the semantic review layer: complementary issues, insights, recommendations, and an independent score prediction. The two score sources, produced by independent mechanisms, can subsequently be compared.

F. System Overview and Design Rationale

Figure 1 shows the architecture. ScoRev couples two components whose division of labor follows directly from the precision–coverage limitation described above, together with a comparison step that relates their outputs.

Deterministic static analysis engine: The engine is the quantitative foundation of the system. Given a Python source file, it computes three scores in the range $[0, 10]$—security, complexity, and maintainability—and produces a structured list of findings, each anchored to a source line and accompanied by supporting evidence. The engine is deterministic: identical input yields identical output, so every assessment is reproducible. The defining property of its design is that no quantity in the scoring procedure is arbitrary. Each constant is either taken from established software-quality literature, adopted as an explicitly stated modeling choice, or declared as a calibratable parameter whose ordering is constrained by evidence; we make this classification explicit for every parameter in Section 5.G. This discipline is what licenses our use of the engine as a reference: its scores are not merely consistent, they are traceable.

Distilled semantic model: The second component is a compact language model fine-tuned to provide precisely what the engine cannot. Through knowledge distillation (Section 5.H), it learns to generate three kinds of semantic output that require reasoning about code intent: complementary issues that the static engine structurally cannot detect, substantive insights that explain the mechanism, consequence, and root cause of a problem, and concrete recommendations. At inference the model is conditioned on the engine's findings: it receives the source code together with the list of issues the engine has already reported, so that its role is to *complement* the deterministic analysis rather than to repeat it. As an illustration, for a web-API source file in which the static engine reports only that documentation and type-annotation coverage are low,

the model identifies a latent runtime fault—a configuration in which disabling the framework's default documentation routes leaves the documentation interfaces unable to retrieve the API schema—a defect that depends on understanding the framework's behavior and the code's intent, and is therefore beyond the reach of pattern-based analysis. We return to this example in later subsections.

The model additionally produces its own prediction of the three quality scores. We treat this as a designed *capability*: the architecture is constructed so that the student can emit semantically informed scores that are anchored to the engine's reference yet adjusted by what the model alone perceives. We do not claim these predicted scores supersede the engine's; we describe the mechanism by which they are produced and bounded.

Independent comparison of the two score sources: Because the engine and the model arrive at their scores by entirely different mechanisms—deterministic metric computation versus learned semantic inference—the two predictions can be compared, providing a principled basis for analyzing where the deterministic and semantic assessments agree and where they diverge.

Design principles: Two principles govern the artifact and recur throughout this section. The first is *grounding*: the static engine emits not only scores but a line-anchored list of findings, and every scoring constant is traceable to a justification. The second is *complementarity*: the language model is trained and operated to add only what static analysis omits, conditioned on the engine's findings so that the two components compose rather than duplicate. Together these principles yield a system whose quantitative foundation is reproducible and auditable and whose semantic layer extends that foundation without inheriting the opacity that makes free-running language models difficult to trust as quality assessors.

G. Static Analysis Engine

The engine maps a Python source file to three scores in $[0, 10]$ together with a line-anchored list of findings. Its design rests on one discipline: every constant in the scoring procedure is traceable. We label each as *grounded* when it is taken from established literature, *justified* when it is a stated modeling choice, and *calibrated* when its ordering is fixed by evidence and its magnitude is tuned against the silver-standard reference of Section 8. We present each dimension as the problem it solves, the principle that informs it, and the resulting formulation. Two properties hold across all three dimensions and motivate the functional forms below: each score mapping is monotonic in the underlying measure, and continuous in it, so that two nearly identical inputs receive nearly identical scores.

a) Quality bands: The $[0, 10]$ scale is partitioned a priori into five quality bands, summarized in Table 2. Every reference point used to derive the scoring parameters of the three dimensions is justified by the band into which it is required to fall, so that the calibrated magnitudes are not free but pinned to a stated quality semantics.

Table 2
QUALITY BANDS FOR THE $[0, 10]$ SCORE SCALE.

Band	Range	Interpretation
Excellent	$[8, 10]$	negligible quality concerns
Good	$[6, 8)$	minor improvements possible
Acceptable	$[4, 6)$	non-trivial issues present
Poor	$[2, 4)$	serious issues requiring revision
Critical	$[0, 2)$	severe defects; not fit for use

1) Security:

a) Problem and principle: Security exhibits an asymmetry that an averaging or cumulative penalty cannot represent: a single confirmed high-severity flaw can compromise a program, while many low-severity observations do not aggregate to the same risk. This is the *weakest-link* principle—system security is bounded by its most severe exposure, not by the mean across findings [25]—and it is the basis of the worst-finding-driven severity ratings used in industrial quality platforms [26]. We further adopt the severity model of the Common Vulnerability Scoring System (CVSS), in which the base severity of a weakness is an intrinsic property, constant across deployment environments [27]. This is exactly the property a static engine can observe, and it fixes the scope of the score precisely: we quantify the intrinsic severity present in the code.

b) Per-finding impact: Let a finding i carry a weakness class cwe_i and a detection confidence c_i. Its impact is the representative CVSS base severity of that class, rescaled to the engine's internal band and weighted by confidence:

$$m_i \;=\; \frac{\mathrm{CVSS}(cwe_i)}{10}\; s\; c_i, \qquad c_i \in \{0.4,\, 0.7,\, 1.0\}, \tag{1}$$

where $\mathrm{CVSS}(cwe_i) \in [0, 10]$ is the representative base score for the weakness class, s rescales the CVSS range onto the impact band, and c_i is the confidence weight for low, medium, and high detection confidence. The per-class base scores are obtained by the established practice of assigning each CWE a representative base severity derived from the vulnerability record [28]; the values used by the engine are listed in Table 3. The confidence factor is what makes a high-severity *class* count as critical only when the detection is itself high-confidence—a high-severity finding reported with low confidence is weighted down accordingly, so the engine does not over-penalize on uncertain pattern matches.

c) Severity-class separation and scoring: Findings split into a critical class and a remainder class at an impact threshold τ:

$$P_{\text{crit}} = \sum_{m_i \geq \tau} m_i, \qquad P_{\text{rest}} = \sum_{m_i < \tau} m_i. \tag{2}$$

The score applies a distinct exponential decay to each class:

$$S_{\text{sec}} \;=\; 10\; e^{-\lambda_1 P_{\text{crit}}}\; e^{-\lambda_2 P_{\text{rest}}}. \tag{3}$$

Exponential decay is the standard functional form for a quantity whose marginal effect diminishes with accumulation,

Table 3
REPRESENTATIVE CVSS BASE SCORES PER CWE CLASS USED BY THE ENGINE [27], [28].

CWE	Description	CVSS base
CWE-78	OS command injection	9.8
CWE-89	SQL injection	9.8
CWE-502	Unsafe deserialization	9.8
CWE-95	Code injection via `eval`/`exec`	9.3
CWE-918	Server-side request forgery	8.6
CWE-327	Use of broken cryptographic primitive	7.5
CWE-798	Hardcoded credentials	7.5
CWE-377	Insecure temporary file	5.5
CWE-676	Dangerous primitive use	3.1

and it admits a half-life reading: each additional unit of penalty multiplies the score by a fixed factor. We exploit this to *derive* both rates from two reference points rather than choosing them. The weakest-link principle requires that a single confirmed critical flaw land in the Critical band of Table 2; we therefore set the reference (p_1, a_1) with a_1 in $[0, 2)$, which yields

$$10\, e^{-\lambda_1 p_1} = a_1 \implies \lambda_1 = \frac{1}{p_1} \ln \frac{10}{a_1}. \tag{4}$$

For the remainder rate we require that an accumulation of moderate findings, collectively non-trivial but not fatal, fall in the Acceptable band; the reference (p_2, a_2) with a_2 in $[4, 6)$ gives $\lambda_2 = \frac{1}{p_2} \ln(10/a_2)$. Pinning each reference to a stated band fixes the asymmetry $a_1 < a_2$ and therefore $\lambda_1 > \lambda_2$ as a consequence of the band ordering, not as a free choice. By construction, (3) cannot rate code containing a confirmed critical vulnerability above code containing only minor findings, however many of the latter are present—the behavior the weakest-link principle demands.

2) Complexity:

a) Problem and principle: Code complexity has two independent facets: the local control-flow structure and the algorithmic cost. A routine can be structurally simple yet algorithmically expensive, so a faithful score must measure both and combine them. For structure we use McCabe's cyclomatic complexity $v(G) = E - N + 2P$, computed from the control-flow graph with E edges, N nodes, and P connected components [29]; the limit of 10 per unit is McCabe's original recommendation and is reaffirmed in NIST SP 500-235 [30], with the moderate, high, and untestable bands at 20 and 50 in standard tooling use. For cost we assign each routine an asymptotic class via the composition rules of algorithm analysis—nested iteration multiplies, sequential composition takes the dominant term, and recursion follows the Master Theorem [31]—and score it on the totally ordered complexity hierarchy $O(1) \prec O(\log n) \prec O(n) \prec O(n \log n) \prec O(n^2) \prec \cdots$, whose order is a mathematical fact rather than a chosen ranking.

b) Structural score: Per-unit values are aggregated for a file by emphasizing its worst unit while retaining the file mean:

$$v_{\text{agg}} = \alpha \max_j v(G_j) + (1 - \alpha)\, \overline{v(G)}. \tag{5}$$

The aggregate maps to a structural score through a continuous, piecewise-exponential function anchored at the McCabe thresholds $t \in \{10, 20, 50\}$ with corresponding anchor scores A_k:

$$S_{\text{struct}}(v) = A_k\, e^{-\lambda_k (v - t_{k-1})}, \quad t_{k-1} < v \le t_k. \tag{6}$$

The anchor scores are chosen to align with the band semantics of Table 2: a unit at McCabe's recommended limit $v = 10$ is acceptable and lands at the top of the Excellent band; a moderately complex unit at $v = 20$ falls in the Good band; a unit at $v = 50$, the boundary at which NIST regards code as untestable, falls in the Poor band; values beyond decay further toward Critical. With the anchors fixed by band membership, continuity is enforced rather than assumed: requiring the segment to meet the next anchor, $A_k e^{-\lambda_k (t_k - t_{k-1})} = A_{k+1}$, determines each rate in closed form,

$$\lambda_k = \frac{1}{t_k - t_{k-1}} \ln \frac{A_k}{A_{k+1}}, \tag{7}$$

so the curve passes exactly through every threshold with no discontinuity. We use this exponential form rather than piecewise-linear interpolation for a concrete reason: a linear mapping changes slope abruptly at each threshold, so two inputs straddling a threshold by one unit can receive disproportionately different scores, whereas (6) crosses the same anchors with a smoothly varying slope and a bounded derivative.

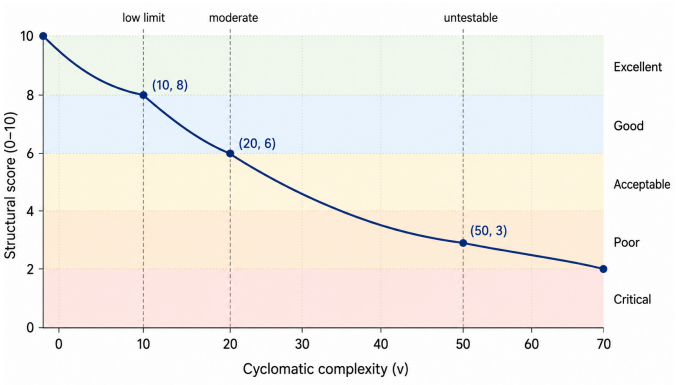

Figure 2. Mapping of cyclomatic complexity to the structural score. The piecewise-exponential function is anchored at McCabe's thresholds $\{10, 20, 50\}$, where the anchor scores $\{8, 6, 3\}$ are pinned to the quality bands of Table 2. The decay rate of each segment is derived in closed form to enforce continuity at every threshold, yielding a smoothly varying slope rather than the abrupt slope changes of a piecewise-linear interpolation.

c) Computational score and fusion: Each asymptotic class maps to a score by a fixed table whose values descend with the hierarchy order. The values are pinned to the band semantics: constant- and logarithmic-time code sits in Excellent, linear and linearithmic in the upper Good band, quadratic in Acceptable, cubic and exponential progressively down through Poor into Critical—the descent in score follows the descent through the bands as the asymptotic class worsens. Time and space classes are combined by taking the worse of the two,

reflecting that the dominant cost governs behavior at scale. The dimension score fuses the computational and structural parts with the computational part weighted higher, since asymptotic cost determines how the routine behaves as input grows while cyclomatic structure is a local property:

$$S_{\text{cpx}} = w\, S_{\text{comp}} + (1 - w)\, S_{\text{struct}}, \qquad w > \tfrac{1}{2}. \tag{8}$$

The cost classifier resolves the asymptotic order of the common algorithmic patterns directly from control-flow structure, following the composition rules of [31].

3) Maintainability:

a) Problem and principle: Maintainability is determined by properties of different kinds—the volumetric size of the code, its semantic readability, and its architectural organization—and no single one subsumes the others. A volumetric index encodes size and control structure but is blind to naming and duplication; a readability measure ignores the size that established indices capture. We therefore compose three complementary layers. The volumetric layer is the Maintainability Index of Coleman, Ash, Lowther, and Oman, a regression-derived combination of Halstead volume, cyclomatic complexity, and program length [32], in its bounded normalization. We keep its cyclomatic term intact: it is a validated component of the index, and the two dimensions sharing this input reflects a genuine relationship between complex and hard-to-maintain code rather than a modeling error. The readability layer rests on empirical evidence linking identifier-naming quality to comprehension and defect-proneness [33], [34] and on the language's documentation and typing conventions [35], [36]; the architectural layer uses duplication density, a standard maintainability indicator in industrial analysis [26].

b) Formulation: The dimension combines the three layers,

$$S_{\text{mnt}} = \beta_1 M + \beta_2 R + \beta_3 T, \tag{9}$$

where M is the normalized Maintainability Index, R combines naming quality, type-annotation coverage, and documentation coverage, and T combines duplication density with a modularity measure. The layer weights β_k are calibrated, but their ordering is fixed by the strength of the evidence behind each: the regression-validated index of Coleman *et al.* carries the dominant weight, and within R the components supported by direct empirical evidence—naming and typing—are weighted above documentation. This last ordering is deliberate: it corrects the common practice of penalizing absent documentation heavily, letting it contribute only in proportion to the comparatively weaker evidence tying it to maintainability. The running example of Section 5.F shows why M alone is insufficient: a file can be volumetrically simple, and thus score well on M, while harboring a latent design fault that only semantic review detects—which is the role of the distilled model introduced next.

H. Grounded Knowledge Distillation

The student model of Section 5.F is trained by distilling the review behavior of a stronger teacher language model into a compact student. Standard practice in distillation transfers the teacher's predictions or rationales to a smaller network [37], [38]; here we adopt this paradigm but constrain it in a way the standard formulation does not: the teacher operates against a deterministic reference produced by the static engine, and the data construction pipeline enforces, at the level of the code rather than at the level of the prompt, that the teacher cannot drift from that reference. We refer to the resulting procedure as *grounded* knowledge distillation. This subsection describes its five mechanisms in turn.

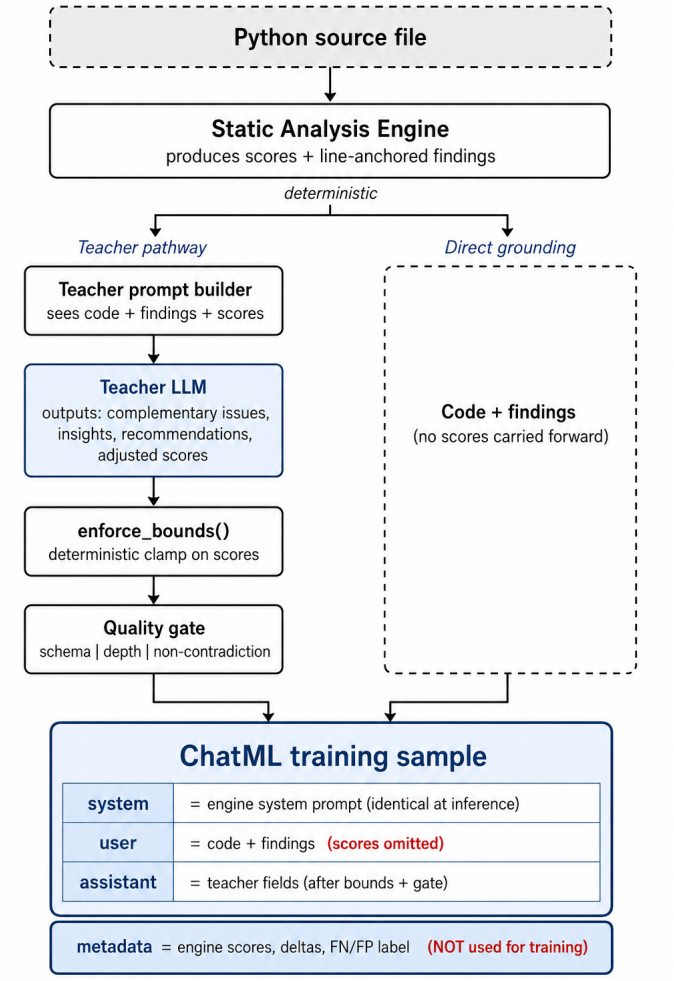

Figure 3. The per-sample grounded distillation pipeline. The teacher pathway observes the engine's scores and produces a four-field semantic output that is then deterministically clamped and gated; the direct grounding path passes only the code and findings forward. The engine's scores enter the metadata but never the assistant target, realizing the asymmetric grounding between teacher and student.

a) 1) Anchored generation against a deterministic reference: For each source file the static engine produces three scores $(s_{\text{sec}}, s_{\text{cpx}}, s_{\text{mnt}})$ and a structured findings list. The teacher receives the numbered source, the engine's findings, and the engine's scores, and is instructed to produce four outputs: complementary issues that the engine could not de-

tect, substantive insights that explain a problem's mechanism, consequence, and root cause, line-anchored recommendations, and a triple of final scores adjusted from the engine's by the teacher's own findings. Two grounding rules are stated explicitly in the teacher's instructions and reinforced by the pipeline: report only what is present in the code, citing the exact line, and never restate an issue already in the engine's findings. The engine's scores therefore serve as an anchor, not a target to imitate; the teacher's role is to contribute the semantic layer the engine cannot reach while remaining tied to the engine's quantitative foundation.

b) 2) Bounded score adjustment enforced in code: A learning-based teacher is free, in principle, to assign any score it likes; in distillation for scoring this is a known failure mode [39]. We remove the freedom rather than rely on the teacher to respect it. The teacher is instructed to deduct from the engine's score only for issues it has actually found, with explicit per-issue and per-dimension caps. Regardless of what the teacher returns, a deterministic post-processing step then enforces, for each dimension $d \in \{\text{sec}, \text{cpx}, \text{mnt}\}$,

$$s_d^{*} \;=\; \text{clamp}_{[0,10]}\Big(\max\big(\, s_d^{\text{eng}} - \Delta_d^{\max},\; \min(s_d^{\text{teach}},\, s_d^{\text{eng}})\big)\Big), \tag{10}$$

where s_d^{eng} is the engine's score for the dimension, s_d^{teach} is the teacher's proposed score, and $\Delta_d^{\max}$ is the per-dimension deduction cap. Equation (10) expresses the rule operationally: the teacher may only *deduct* from the engine ($s_d^{\text{teach}} \le s_d^{\text{eng}}$) and may not deduct more than $\Delta_d^{\max}$ from any single example. This guarantees that the labels used to train the student are bounded with respect to the deterministic reference, no matter how the teacher behaves on a given input.

c) 3) Asymmetric grounding between teacher and student: A pitfall of distillation for scoring is for the student to learn to copy the teacher's number rather than to derive it from the code. We prevent this by making the inputs visible to the teacher and to the student deliberately different. The teacher sees the source, the engine's findings, and the engine's scores, so that it can anchor its output. The student, by contrast, sees only the source and the engine's findings; the engine's scores are *omitted* from the user message of the training sample. The training target therefore requires the student to infer a score from the code and the findings, not to copy a number that is present in the prompt. The engine's scores remain in the per-sample metadata and are used for analysis but never exposed during training or inference.

d) 4) Multi-layer quality gate: Before a sample is admitted to the training set, the pipeline applies a deterministic gate that rejects three failure modes. (i) Schema failure: the teacher's response must parse into the expected fields and types; otherwise the sample is dropped. (ii) Shallow reasoning: the insights field must contain substantial content, measured in words, so that the distilled rationale carries the mechanism-consequence-cause analysis the prompt requests rather than a restatement of the issue. (iii) Internal contradiction: the teacher's adjusted scores must be consistent with its own findings—for example, a dimension cannot be deducted from while the teacher reports no issue in that dimension, nor may the score rise above the engine's reference. The gate is applied per sample, not as a post-hoc batch filter, so its rejections leave no trace in the final data; we note its existence here as a property of the construction procedure.

e) 5) Engine self-assessment as a by-product: A consequence of (10) is that each admitted sample carries, in its metadata, a comparison between the engine's scores and the bounded teacher's scores: the per-dimension delta $\delta_d = s_d^{\text{eng}} - s_d^{*}$ and the list of complementary issues the teacher contributed. From these we derive a per-sample engine-assessment label: ACCURATE when no deduction was applied and no complementary issue was contributed; FALSE_NEGATIVE when the teacher contributed at least one material complementary issue and a substantive deduction; and FALSE_POSITIVE when the engine's finding was downgraded by the teacher. The label is metadata only and is never used as a training signal—it is an artifact of the same code that produces the training data. It nonetheless gives a principled view of where the deterministic engine systematically misses semantic defects, a view we will report on in Section 8.

f) Per-sample output and dataset assembly: A sample admitted by the per-sample pipeline is stored in ChatML form. The *system* message is the same prompt the engine uses at inference, so that training-time and inference-time conditioning are identical. The *user* message contains the numbered source and the engine's findings—and only these. The *assistant* message is the teacher's four-field output after the bounded clamp of (10) and the per-sample quality gate. The engine's original scores, the per-dimension deltas, and the engine-assessment label live in a separate metadata block that does not participate in training. Across the four data-collection streams, samples are then merged, deduplicated by a content hash of the user message, and passed through a stricter post-collection gate that raises the insight-depth threshold and rejects trivial files and internally inconsistent samples. From the same metadata we classify each remaining sample's engine-assessment label by severity: MATERIAL when a substantive deduction was applied or a real security-class issue was contributed, MINOR when only a small deduction was applied, and NONE otherwise—a deliberate separation that prevents the raw complementary-issue rate from inflating the engine's miss rate. The result is a corpus of $8{,}761$ samples, partitioned $95/5$ into training and validation sets, stratified by code category. The running example introduced in Section 5.F is, under this construction, a MATERIAL FALSE_NEGATIVE: the engine's findings note only that documentation and typing coverage are low, while the teacher contributes the latent design fault and the corresponding small, bounded deduction.

This dataset is the artifact that connects the deterministic engine to the semantic student. The procedure that produces it is part of the system under study, and its mechanisms are the means by which a free-form teacher language model is converted into a controlled source of supervision that the small student can safely learn from.

I. Model Fine-Tuning

a) Base model: We adopt Qwen2.5-3B-Instruct [40] as the student. The choice is deliberate: at three billion parameters the model is small enough to fine-tune and serve on a single consumer-grade GPU while remaining capable enough, in its instruction-tuned form, to follow the structured review schema introduced in Section 5.H. The instruction-tuned variant matters: the training data is delivered in conversational ChatML form, and a base model without instruction-following pretraining would require a substantially longer adaptation phase to learn the format before learning the task.

b) Parameter-efficient fine-tuning: Full fine-tuning of a three-billion-parameter model is impractical on commodity hardware and unnecessary for a task that can be expressed as an adaptation of existing competence. We use Quantized Low-Rank Adaptation (QLoRA) [41], which combines the low-rank adapter formulation of Hu *et al.* [42] with 4-bit NF4 quantization of the frozen base weights and double quantization of the quantization constants. The pre-trained weights are loaded in NF4; the LoRA adapters, kept in higher precision, are the only parameters updated by backpropagation. This reduces the trainable-parameter footprint by orders of magnitude relative to the base model and makes single-GPU fine-tuning feasible without sacrificing the precision of the task-specific updates.

c) Training objective: Each sample is the three-turn ChatML structure assembled in Section 5.H: a fixed *system* message identical to the one used at inference, a *user* message containing the numbered source and the engine's findings, and an *assistant* message carrying the bounded teacher output. We train with the supervised fine-tuning loss masked to the assistant turn only: the cross-entropy is computed exclusively over the assistant tokens, while the system and user tokens are visible as context but excluded from the loss. This focuses the gradient signal on what the student is meant to produce and prevents it from being penalized for the fixed scaffolding it should condition on rather than reproduce.

d) Hyperparameters and schedule: Table 4 lists the configuration. Three choices warrant a specific justification. *First*, we fine-tune for a single epoch. Single-epoch QLoRA fine-tuning on instruction-style data has been reported to match or outperform multi-epoch training on the same corpus [43], [41], an observation we adopt rather than contradict. *Second*, we use a per-device batch size of one with gradient accumulation of eight, yielding an effective batch of eight; this is the standard accommodation when the per-step memory budget is dominated by the maximum sequence length, and it preserves the effective batch a small-model cosine schedule expects [41]. *Third*, the maximum sequence length is set so that the longest assembled ChatML samples fit without truncation; truncating the assistant turn would silently corrupt the training target, and the dataset construction filters of Section 5.H already restrict source length to keep this guarantee. Training was performed on a single NVIDIA T4 GPU (16 GB) hosted on Kaggle; total training time was on the order of one wall-clock day.

e) Validation protocol: The held-out validation split established in Section 5.H contains five percent of the corpus, stratified by code category to preserve the category mix of the training set. The training loss and the validation loss are monitored throughout fine-tuning; we report only the final fine-tuned model and reserve quantitative results to Section 8. The artifact this subsection produces—the fine-tuned Qwen2.5-3B-Instruct adapter, applied at inference on top of the quantized base model—is the second of the two collaborating components introduced in Section 5.F. Together with the deterministic engine of Section 5.G, it constitutes the complete ScoRev system.

Table 4
FINE-TUNING CONFIGURATION.

Component	Value
Base model	Qwen2.5-3B-Instruct
Adaptation method	QLoRA, NF4 with double quantization
LoRA rank r	16
LoRA α	16
LoRA dropout	0.0
LoRA target modules	q, k, v, o, gate, up, down projections
Optimizer	AdamW (paged, 8-bit)
Learning rate	2×10^{-4}
LR scheduler	cosine
Warm-up steps	10
Per-device batch size	1
Gradient accumulation	8 (effective batch 8)
Maximum sequence length	4096 tokens
Epochs	1
Precision	bf16 compute
Loss masking	assistant turn only
Hardware	1× NVIDIA T4 (16 GB), Kaggle

6. LOCATION AND SAFETY CONSIDERATIONS

J. Development, Training, and Deployment Locations

The development, dataset construction, model training, and deployment phases of ScoRev were carried out across multiple computational environments that were designed to address the needs of each project stage. This distributed infrastructure allows us to make the most of the resources available and maintain reproducibility and scalability through the lifespan of the research.

1) Development and Data Processing Environment: The data collection, preprocessing, telemetry generation, and dataset construction stages were conducted mainly in cloud-based notebook environments, especially Google Colab. These environments were used for:

- Repository acquisition from public open-source sources.
- Dataset construction and curation.
- Data preprocessing and filtering.
- Telemetry generation.
- Semantic enrichment orchestration.
- Structural validation and consistency checks.

In addition, local development machines operated by team members were used for software implementation, debugging, validation, and auxiliary development tasks.

To speed up large dataset building, the project team consisted of four members, and repository collections were distributed according to a specific stratification strategy, allowing

Table 5
LOCAL DEPLOYMENT HARDWARE CONFIGURATION

Component	Specification
GPU	RTX 4060 8GB

for parallel processing of repository subsets while maintaining dataset diversity.

The semantic enrichment stage also used the Gemini 2.5 Flash Lite API to generate:

- Structured reasoning.
- Engine reliability classifications.
- Issue justifications.
- Remediation recommendations.

2) Training and Fine-Tuning Environment: The language model training phase was performed using Kaggle Notebooks, which provided cloud-based GPU resources for parameter-efficient fine-tuning.

The student model was based on Qwen2.5-3B-Instruct and fine-tuned using the QLoRA methodology, which allows for efficient adaptation with limited computational resources.

Kaggle infrastructure was used for:

- Training execution.
- Experiment management.
- Checkpoint storage.
- Model artifact management.

Despite the limitations of academic-tier cloud resources and the absence of large-scale dedicated GPU infrastructure, the training process generated stable adapter checkpoints that would be made available for downstream use. The final adapter was then published on the Hugging Face Hub to ensure reproducibility and transparency for future research.

3) Deployment and Inference Environment: First deployment experiments were made on Microsoft Azure cloud infrastructure.

Due to academic subscription constraints and low GPU availability, inference workloads on Azure were executed with CPU-based resources. The Azure deployment was primarily used for:

- Functional validation.
- Integration testing.
- Compatibility assessment.
- Deployment verification.

The full ScoRev system was also deployed on a local workstation owned by one of the members of the project for operational evaluation and end-to-end testing.

The local deployment environment was used for:

- Real-time inference.
- Graphical User Interface (GUI) execution.
- Latency monitoring.
- Debugging and diagnostics.
- End-to-end validation.

K. Safety, Security, and Ethical Considerations

Several measures have been taken for safety, integrity, and responsible use of the ScoRev framework.

1) Use of Public Data Sources: All repositories used for dataset building were publicly available open-source sources. No proprietary software, private repositories, confidential source code, or personally identifiable information (PII) were intentionally collected or processed during the study.

2) Secure Processing and Storage: The dataset generation pipeline only processed source code artifacts for research purposes. Intermediate results, telemetry, model checkpoints, and datasets were stored in controlled cloud environments and project-owned storage resources.

3) Responsible AI Usage: The language model component provides software quality assessment and code review support and recommendations for fixing it. The system does not automatically modify source code, deploy software, or act on external recommendations. Human oversight is required for all engineering decisions based on the recommendations.

4) Research Reproducibility: To improve transparency and reproducibility, the trained adapter was published through the Hugging Face Hub, allowing independent researchers to reproduce the inference workflow and evaluate the model under different deployment environments.

5) Deployment Safety: ScoRev operates as a static-analysis and review framework and does not run the analyzed source code. This design is meant to reduce the risk of running potentially unsafe software during analysis and to evaluate the code artifacts in an isolated and controlled manner.

L. Future Infrastructure Considerations

The architecture of ScoRev was deliberately designed to be cloud portable. The deployment pipeline is not tied to a specific cloud provider and can be migrated to other infrastructures supporting modern machine learning workloads.

Future deployments may leverage dedicated GPU resources to improve inference latency and throughput while preserving compatibility with the existing Azure-based validation environment.

The availability of the trained adapter through Hugging Face further facilitates future cloud deployment, collaborative research, and long-term maintainability of the framework.

7. EXPERIMENTAL SETUP

M. Overview and Research Questions

The architectural rationale of Section 5.F and the gap identified in Section 4.E motivate five research questions, each addressed by a dedicated experiment introduced in Sections 7.N–7.Q and evaluated in Sections 8.S–8.W. Following the design-and-evaluation-of-a-particular-instance paradigm for software engineering research [44], each question is framed as a qualified or existential claim about ScoRev as a specific system, rather than as a global claim about code-quality assessment in general.

a) RQ1: How does ScoRev's deterministic, literature-grounded scoring compare to an emerging silver standard of multi-LLM-judge consensus, and what does the stability of that consensus itself reveal about the viability of LLM-based assessment as a validation reference?

b) RQ2: To what extent does grounding a fine-tuned model in deterministic static-analysis findings improve defect-detection effectiveness beyond what either component achieves independently?

c) RQ3: To what extent can a locally-deployed, privacy-preserving hybrid system match the defect-detection capability of a commercial frontier-scale language model?

d) RQ4: Do students trained through grounded distillation develop an internalized, semantically-enriched scoring judgment that converges with the Engine — without accessing its numeric output at any stage?

e) RQ5: Does ScoRev produce an internally consistent, monotonically ordered signal across a designed severity gradient, and does this monotonicity provide preliminary evidence that ScoRev's scores could serve as a directional feedback signal in iterative, AI-assisted code-refinement workflows?

RQ1 is addressed by the Silver-Standard experiment (E0, Section 7.N); RQ2 and RQ3 by the Detection Benchmark (Section 7.O); RQ4 by the Cross-Validation experiment, which reuses the Detection Benchmark's corpus (Section 7.P); and RQ5 by the Ranking experiment (Section 7.Q). Results are reported in the correspondingly numbered subsections of Section 8.

N. Silver-Standard Setup

The Silver-Standard experiment (E0) evaluates RQ1 by comparing the Engine's deterministic scores (Section 5.G) against an external, LLM-judge-based consensus on a held-out corpus.

The corpus comprises $N{=}120$ Python files, stratified across the five categories used in dataset construction (Section 5.H: clean, complex, security-sensitive, machine-learning, and legacy), with 24 files per category. None of these files overlap with the fine-tuning dataset of Section 5.H.

Three frontier-scale large language models (one each from the GPT, Claude, and Gemini model families) independently score each file on the same security/complexity/maintainability rubric (0–10), following the LLM-as-judge protocol established for general-purpose model evaluation [45]. For each dimension, the consensus reference is the mean of the three judges' scores. Agreement among the three judges — the *inter-judge ceiling* — is computed as the mean pairwise Spearman correlation across all three judge pairs, per dimension, and serves as an upper bound against which the Engine–consensus correlation can be interpreted.

Convergence between the Engine and the consensus is reported via Spearman's rank correlation coefficient ρ and mean absolute error (MAE) over all $N{=}120$ files, per dimension, in Section 8.S. As a secondary view, per-category correlations (24 files per category) are reported where they are informative; with $N{=}24$, these category-level estimates carry wide confidence intervals and are interpreted accordingly in Section 8.X.

A second, dataset-scale view of Engine accuracy is available as a by-product of the grounded knowledge distillation process (Section 5.H). During dataset construction, the teacher model classifies each Engine finding, per dimension, as accurate or as a false positive, and classifies any material discrepancy between the Engine's score and its own assessment as a false negative of bounded magnitude (Eq. 10). Aggregated over all 8,761 training samples, this yields dataset-wide rates of Engine accuracy, false positives, and false negatives, reported alongside the $N{=}120$ correlations in Section 8.S.

O. Detection Benchmark Setup

The Detection Benchmark evaluates RQ2 and RQ3 by measuring how accurately the semantic layer identifies known issues in a small, manually annotated corpus, under four system configurations that isolate the contributions of static-analysis grounding, fine-tuning, and model scale.

1) Benchmark corpus: The corpus consists of ten Python files (S1–S10), summarized in Table 6. The files were designed, and their issues annotated, before any model was run against them, to avoid post-hoc selection. Across the eight non-clean files, 29 issues are annotated in total; S5 and S10 contain zero annotated issues and serve as false-positive probes.

2) Systems compared: Four configurations are evaluated on the same corpus, isolating the two architectural components under test — static-analysis grounding and fine-tuning — against a frontier commercial reference. All locally-run configurations (A, B, C) use 4-bit NF4 quantization, greedy decoding (`do_sample=False`), and a 2,500-token generation budget.

A — ScoRev (full system). The fine-tuned `qwen-scorev-v2` model receives the Engine's system prompt (Section 5.G, verbatim) and a user message constructed by `build_user_message`: line-numbered source code together with the Engine's real findings, formatted as a delimited block. This is the inference-time configuration of the full hybrid pipeline (Section 5.F).

B — grounding ablation. Identical to A in every respect except that the findings block is replaced with the empty placeholder (`"(none)"`). This isolates the marginal contribution of static-analysis findings at inference time, holding the model and all other inputs fixed.

C — fine-tuning ablation. Identical input to A (line-numbered code with real findings) but processed by the vanilla, non-fine-tuned `Qwen2.5-3B-Instruct` checkpoint. This isolates the marginal contribution of grounded fine-tuning, holding the input format and model scale fixed.

E — frontier reference. Gemini 3 Flash-Lite, queried through its general-purpose web interface with a prompt that requests the same issue/insight/recommendation/score schema but supplies neither line numbers nor Engine findings. E represents an ungrounded, frontier-scale point of comparison for RQ3.

Table 6
DETECTION BENCHMARK CORPUS (S1–S10). ISSUE COUNTS ARE THE NUMBER OF INDEPENDENTLY ANNOTATED ISSUES PER FILE; CLEAN FILES (S5, S10) CONTAIN ZERO ANNOTATED ISSUES AND ARE USED TO PROBE FOR FALSE POSITIVES.

Sample	Category	#Issues	Representative issue types
S1	Balanced	5	SQL injection, $O(n^5)$ nested loop, insecure deserialization (`pickle`), resource leak, missing docs/types
S2	Security-heavy	4	Hardcoded credential, command injection (`shell=True`), weak hash (MD5), unescaped HTML (XSS)
S3	Complexity-heavy	3	$O(n^3)$ nested loop, exponential recursion (no memoization), deeply nested conditionals
S4	Maintainability-heavy	4	Non-descriptive naming, duplicated magic-number logic, missing docs/types, unguarded division
S5	Clean (1)	0	Fully documented, type-hinted; false-positive probe
S6	Balanced	3	Unsafe `yaml.load`, $O(n^2)$ string concatenation, mutable shared class state
S7	Security-heavy	3	Insecure randomness for tokens, path traversal, redundant boolean branch
S8	Complexity-heavy	3	$O(n^2)$ bubble sort, nested-loop matrix construction, unbounded `while True`
S9	Maintainability-heavy	4	Non-descriptive parameters, manual date formatting, `== None` comparison, missing docs/types
S10	Clean (2)	0	Fully documented, type-hinted; false-positive probe

3) Matching protocol: For each annotated issue, a manual judgment of true positive (TP), false positive (FP), or false negative (FN) is made against the system's combined output (the union of all reported issues, complementary issues, insights, and recommendations, regardless of which output field or dimension label they appear under). A system output counts as a TP if it identifies the same underlying problem at the same code location as an annotation, irrespective of the `dimension` label assigned to it; per-issue dimension labels are known to be unreliable across all evaluated models (Section 5.H) and are therefore not used as a matching criterion. An FP is any reported issue with no corresponding annotation. An FN is any annotation with no corresponding system output. For the two clean files (S5, S10), any reported issue is counted as an FP and recall is undefined. Precision, recall, and F_1 are computed in the standard way from the resulting TP/FP/FN counts, aggregated over all ten files.

P. Cross-Validation Setup

The Cross-Validation experiment evaluates RQ4 by reusing the S1–S10 corpus of Section 7.O and comparing, for each file and each dimension, the Engine's deterministic score against the score independently predicted by `qwen-scorev-v2`.

The model's score prediction is produced by the second, *blind* call of the asymmetric two-call architecture (Section 5.F): the model receives the code and the Engine's findings but not the Engine's numeric scores, and is prompted to predict security, complexity, and maintainability scores on the same 0–10 scale. This is the same configuration as Run A (Section 7.O), reusing its outputs; no additional inference is required.

For each dimension, convergence between the Engine's score and the model's predicted score is reported via Spearman's ρ over the ten files, together with the per-file Cross-Validation Agreement value $\text{agreement} = 1 - \text{mean}_d(|s_d^{\text{engine}} - s_d^{\text{model}}|/10)$ (the agreement formula), summarized by its mean and standard deviation across the ten files.

Q. Ranking Benchmark Setup

The Ranking experiment evaluates RQ5 by measuring whether ScoRev's scores change monotonically along a controlled severity gradient.

Five Python files (L1–L5) implement variants of the same underlying functionality, ordered by design so that each successive level introduces issues belonging to a higher severity tier than the level before it. The five tiers correspond directly to the five quality bands of Table 2: L1 is exemplary (fully documented, type-hinted, no findings of any kind); L2 introduces only maintainability-tier issues (undocumented parameters, a non-descriptive identifier) with no CWE-mapped weakness; L3 introduces the first CWE-mapped security weakness; L4 and L5 successively introduce additional, more severe CWE-mapped weaknesses together with structural complexity issues. Construction is tier-cumulative rather than line-for-line cumulative: each level is designed to instantiate its target tier of Table 2, not merely to append code to the previous level.

For each of the four scored quantities (security, complexity, maintainability, and the overall weighted score, the overall score with the default importance weights of the UI), monotonicity across L1–L5 is assessed via Spearman's ρ and, given $N{=}5$, corroborated with Kendall's τ where the resulting p-value is informative. Each level's set of findings is additionally cross-referenced against the CWE-to-severity mapping of Table 3 to verify that the intended severity escalation corresponds to the intended CWE entries.

8. RESULTS

R. Illustrative Walkthrough

Figure 4 illustrates the complete output of the ScoRev pipeline for a representative Python file, concretizing the three-layer architecture of Section 5.F: the Engine's scores serve as the primary quantitative signal, the semantic layer contributes complementary diagnostics and actionable recommendations, and the cross-validation mechanism produces a confidence indicator over the two score sources.

As a concrete instance of the security formula's weakest-link behaviour (Eq. 3): S1 (Table 6) contains both a SQL-injection pathway and an unguarded `pickle.load` call, each independently mapping to a CWE entry in Table 3. The Engine reports $S_{\text{sec}} = 0.0$ for this file, the minimum admissible value, reflecting the co-occurrence of two independently CWE-mapped weaknesses rather than the score

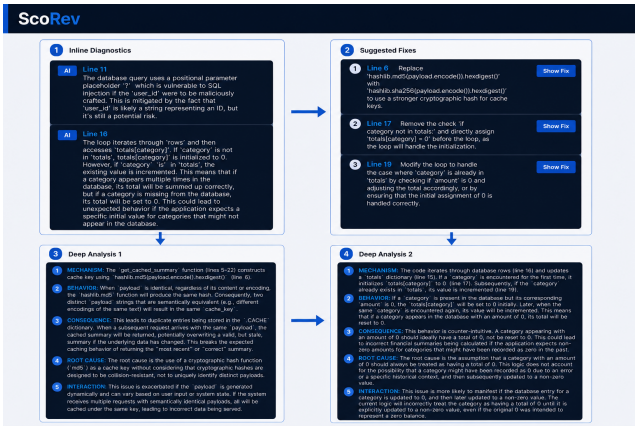

Figure 4. ScoRev output for a representative input file. (1) Inline diagnostics with line-anchored issue identification. (2) Actionable fix suggestions. (3)–(4) Structured deep analyses following the five-component diagnostic format of Section 5.H: mechanism, behaviour, consequence, root cause, and interaction.

of either weakness in isolation. The semantic layer's treatment of the `pickle.load` call follows the five-component diagnostic structure of Section 5.H (mechanism, behaviour, consequence, root cause, interaction), and its recommendation replaces the unsafe deserialization call with a safe alternative, in the same style as the corrected-cryptography and corrected-deserialization recommendations of Section 8.U.

S. RQ1: Silver-Standard Convergence

The Engine's scores correlate with the three-judge consensus to a degree that is itself bounded by how much the three judges agree with *each other*. Table 7 reports, per dimension, the Engine–consensus Spearman correlation ρ, the mean absolute error, and the inter-judge ceiling over $N{=}120$ files.

Table 7
SILVER-STANDARD CORRELATIONS (E0, $N{=}120$). THE INTER-JUDGE CEILING IS THE MEAN PAIRWISE SPEARMAN CORRELATION AMONG THE THREE LLM JUDGES AND BOUNDS THE ENGINE–CONSENSUS CORRELATION FROM ABOVE.

Dimension	Engine–consensus ρ	MAE	Inter-judge ceiling
Security	0.286	1.531	0.561
Complexity	0.559	1.242	0.647
Maintainability	0.321	2.328	0.394

For complexity, the Engine's correlation with the consensus (0.559) reaches 86% of the inter-judge ceiling (0.647); for maintainability, it reaches 81% of a substantially lower ceiling (0.321/0.394); for security, it reaches 51% of the ceiling (0.286/0.561). In every dimension, the inter-judge ceiling itself is well below 1.0, indicating that the three frontier-scale judges do not agree with each other much more than the deterministic Engine agrees with their consensus.

This pattern is consistent with documented properties of LLM-based judging. Pairwise agreement among LLM judges has been shown to fall substantially below perfect agreement even among frontier models [46], and LLM judges have been shown to systematically score more leniently than human expert panels [47]. For maintainability specifically, the low absolute ceiling (0.394) is consistent with maintainability being the most subjective of the three dimensions even for human raters: in a study spanning 70 professional developers across 17 companies, human raters agreed with a consensus maintainability judgment only 70% of the time, with substantial disagreement on 17% of cases [48]. The conservativeness of Maintainability-Index-based metrics relative to LLM judgments has separately been noted in the literature on which the maintainability formulation of Section 5.G3 is grounded [20].

At the category level (24 files per category), one reversal is notable: for the *legacy/mixed* category, the Engine–consensus security correlation is negative ($\rho \approx -0.207$). This single category-level estimate is discussed in Section 8.X together with the sample-size considerations that bound its interpretation; the aggregate $N{=}120$ correlation of Table 7 remains the primary estimate.

1) Dataset-wide engine-assessment rates: A second, much larger-scale view of Engine accuracy is available from the grounded knowledge distillation process itself (Section 5.H), which records, for every one of the 8,761 training samples and each scoring dimension, whether the teacher model accepted the Engine's score as accurate, identified a bounded false negative (a missed issue, classified as minor or material according to whether the resulting score adjustment reached the bound $\Delta_{\max}$ of Eq. 10), or flagged an existing Engine finding as a false positive. Table 8 reports these rates, aggregated over all 8,761 samples.

Table 8
DATASET-WIDE ENGINE SELF-ASSESSMENT RATES ($N{=}8{,}761$), DERIVED AS A BY-PRODUCT OF GROUNDED KNOWLEDGE DISTILLATION (SECTION 5.H). FP RATE IS THE FRACTION OF SAMPLES FOR WHICH THE TEACHER FLAGGED AN ENGINE FINDING AS A FALSE POSITIVE; MINOR FN AND MATERIAL FN ARE CLASSIFIED BY WHETHER THE TEACHER'S SCORE ADJUSTMENT REACHED THE BOUND $\Delta_{\max}$ OF EQ. 10.

Dimension	Accurate	Minor FN	Material FN	FP rate
Security	95.5%	4.5%	0.0%	0.0%
Complexity	90.3%	9.6%	0.0%	0.0%
Maintainability	38.0%	61.2%	0.8%	0.0%

The Engine's security and complexity assessments are accepted by the teacher as accurate on 95.5% and 90.3% of samples respectively, with all deviations classified as minor false negatives and no false positives recorded in either dimension. Maintainability is the dimension where the Engine's deterministic formulation is least sufficient: the teacher identifies a scoring gap on 62.0% of samples, overwhelmingly minor in severity (61.2%) with material gaps on 0.8%. This distribution directly motivates the hybrid architecture of Section 5.F: the semantic layer adds the most complementary value precisely

Table 9
AGGREGATE DETECTION PERFORMANCE OVER THE TEN-FILE DETECTION BENCHMARK (29 ANNOTATED ISSUES ACROSS S1–S4 AND S6–S9; S5 AND S10 CONTRIBUTE TO FP COUNTS ONLY).

Run	TP	FP	FN	Precision	Recall	F_1
A — ScoRev (full)	17	8	12	0.680	0.586	0.630
B — grounding ablation	11	10	18	0.524	0.379	0.440
C — fine-tuning ablation	12	6	17	0.667	0.414	0.511
E — frontier reference	26	0	3	1.000	0.897	0.945

Table 10
PER-FILE F_1 FOR RUNS A, B, AND C. FOR THE CLEAN FILES (S5, S10), NO ANNOTATED ISSUES EXIST; THE VALUES SHOWN ARE FP COUNTS, NOT F_1.

Sample	Category	F_1^A	F_1^B	F_1^C
S1	Balanced	0.444	0.250	0.571
S2	Security	0.750	0.333	0.667
S3	Complexity	0.800	0.333	0.571
S4	Maintainability	0.400	0.400	0.400
S5	Clean	1 FP	1 FP	0 FP
S6	Balanced	0.333	0.800	0.400
S7	Security	0.800	0.800	0.400
S8	Complexity	0.800	0.800	0.800
S9	Maintainability	1.000	0.286	0.400
S10	Clean	2 FP	2 FP	2 FP

where the deterministic Engine is least complete, without displacing it where it is already accurate.

Figure 5 presents the Engine–consensus relationship of Table 7 as a 1×3 scatter, in the same visual style as Figure 6 (Section 8.V): the two figures are intended to be read as a pair, with the spread around the identity line here contrasting with its absence there.

T. RQ2: Detection Ablation — Contribution of Static-Analysis Grounding

Grounding the fine-tuned model in the Engine's findings at inference time improves its detection F_1 by 0.190 in absolute terms, a 43% relative improvement (Table 9, rows A and B). The improvement is driven predominantly by recall (+0.207) rather than precision (+0.156): findings act primarily as a coverage signal, directing the model toward issues it would otherwise miss, rather than suppressing spurious reports.

The largest A–B gap occurs on S9 (F_1^A=1.000 vs. F_1^B=0.286): without the Engine's findings, the ungrounded model reduces to generic stylistic commentary and misses the file's maintainability-specific issues entirely. S6 is the single sample on which B exceeds A (0.800 vs. 0.333); rather than indicating that findings are harmful, this is evidence that the relationship between grounding and detection is complementary rather than purely additive — on this particular file, the findings block appears to narrow the model's attention away from an issue it would otherwise have reported unprompted.

U. RQ3: Frontier Comparison — Grounding versus Scale

Fine-tuning on grounded data improves detection F_1 by 0.119 in absolute terms over the vanilla 3B model given the same grounded input, a 23% relative improvement (Table 9, rows A and C). As with the grounding ablation of Section 8.T, the improvement is concentrated in recall (0.414 → 0.586) rather than precision (0.667 → 0.680): fine-tuning teaches the model to act on the findings it is given, rather than merely to restate them.

The largest A–C gap again occurs on S9 (F_1^A=1.000 vs. F_1^C=0.400): the vanilla model's three responses reduce to a generic "lacks a docstring" comment, whereas the fine-tuned model identifies the file's specific naming and date-formatting issues. On S7, A identifies both annotated security issues (replacing an insecure random-number call with `secrets.token_hex`, and flagging the path-traversal vulnerability), while C identifies neither.

Against the frontier reference, ScoRev recovers 67% of Gemini 3 Flash-Lite's detection F_1 (0.630/0.945) using a 3B-parameter model that runs locally under 4-bit quantization (Table 9, rows A and E). All three of Gemini's false negatives are of a single type — absence of docstrings or type hints, occurring on S1, S4, and S9 — so the remaining gap on this benchmark is concentrated specifically in documentation-completeness reasoning, a single issue type falling under the maintainability dimension. The 67% recovery is achieved without any external API dependency, recurring inference cost, or transmission of source code to a third party, the trade-off that Section 4.E identifies as the motivation for a locally-deployable grounded model.

1) Qualitative recommendations: Across the benchmark, A's recommendations correctly target the underlying weakness rather than its symptoms: on S6, the unsafe `yaml.load` call is replaced with `yaml.safe_load` (the CWE-502 mitigation of Table 3); on S3, the exponential recursive Fibonacci implementation is replaced with an iterative formulation, directly addressing the file's complexity finding; on S2, `hashlib.md5` is replaced with `hashlib.sha256`; and on S9, manual string-concatenation date formatting is replaced with `datetime.strftime`, alongside a separate recommendation replacing an `== None` comparison with the idiomatic `is None` form.

2) Score consistency on representative files: For S1, which contains both a SQL-injection pathway and an unguarded `pickle.load` call, A's predicted security score is 0.0/10. For the two clean files (S5, S10), A's predicted scores are 10.0/10 across all three dimensions. This pairing — a maximally penalized score on a file containing two independently CWE-mapped vulnerabilities, and a maximally rewarded score on files containing none — is the pattern required for the scores to function as a quality-assurance signal for generated code, the motivating use case of Section **??**.

V. RQ4: Cross-Validation Convergence

The model's blindly predicted scores (Section 7.P) converge strongly with the Engine's deterministic scores across all three dimensions (Table 11, Figure 6), with the mean per-file Cross-Validation Agreement at 0.950 (σ=0.057) over the ten files.

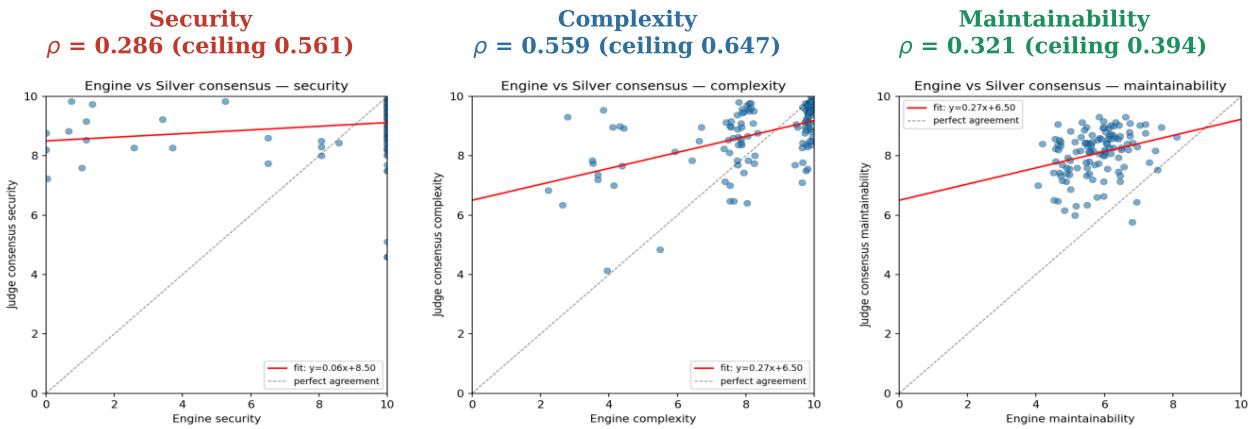

Figure 5. Engine scores versus three-judge consensus (E0, N=120), per dimension. The dashed line denotes perfect agreement. Compare with Figure 6 (Section 8.V), which presents the Engine–model relationship at the same scale.

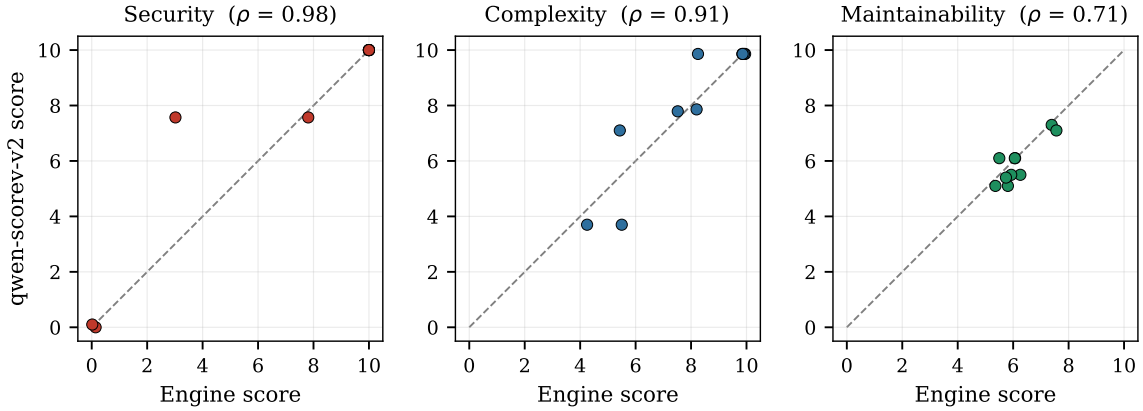

Figure 6. Engine versus model-predicted scores across the ten-file Detection Benchmark, per dimension. The dashed line denotes perfect agreement. Compare with Figure 5 (Section 8.S), which presents the Engine–consensus relationship at the same scale.

Table 11
ENGINE–MODEL SCORE CORRELATIONS (CROSS-VALIDATION, N=10).

Dimension	ρ
Security	0.981
Complexity	0.915
Maintainability	0.710

Maintainability shows the weakest, though still strong, convergence (ρ=0.710) and the lowest single-file agreement, on S6 (0.794). On S6, the Engine's security score reflects its single unguarded `yaml.load` call (Table 6), while the model's blindly predicted security score is higher; this is the same direction of residual leniency observed in the maintainability and security dimensions of the Silver-Standard comparison (Section 8.S), and is consistent with the student model having partially inherited the teacher's leniency bias along with its scoring function.

W. RQ5: Ranking and Monotonicity

Across the five-level severity gradient L1–L5 (Section 7.Q), ScoRev's overall score and its maintainability component are perfectly monotonically decreasing ($\rho = -1.000$ for both), and its security and complexity components are monotonically decreasing with a single exception each (Table 12, Figure 7).

Table 12
RANK CORRELATIONS BETWEEN SEVERITY LEVEL (L1–L5) AND EACH SCORED QUANTITY (N=5). FOR $\rho = \pm 1.000$, THE EXACT TWO-TAILED p-VALUE AT N=5 IS $1/120 \approx 0.0083$; THE VALUES SHOWN FOR SECURITY AND COMPLEXITY ARE AS REPORTED BY STANDARD IMPLEMENTATIONS.

Quantity	ρ	p	τ
Security	-0.975	≈ 0.005	-0.949
Complexity	-0.900	≈ 0.037	-0.800
Maintainability	-1.000	$1/120$	—
Overall	-1.000	$1/120$	—

L1 and L2 both yield S_{sec}=10.0, reflecting the absence of any CWE-mapped weakness in either implementation.

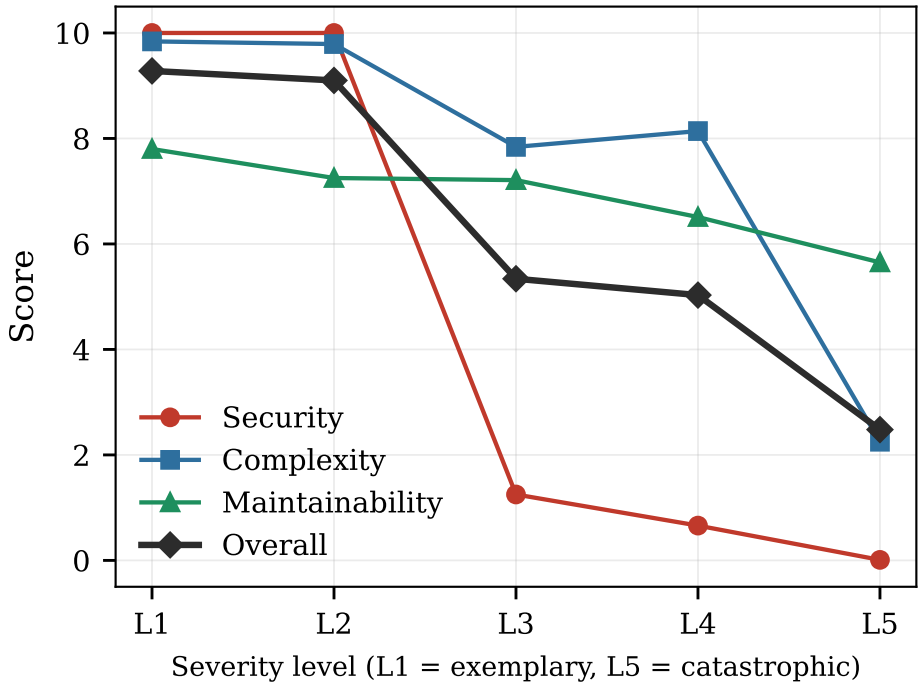

Figure 7. Security, complexity, maintainability, and overall scores across the five-level severity gradient L1 (exemplary) to L5 (catastrophic).

The hardening gaps introduced in L2 — unsalted hashing and a non-constant-time comparison — fall outside the CVSS-anchored taxonomy underlying Eq. 3 and are correspondingly surfaced not by the Engine but by the semantic layer (Section 5.H), which reports them as complementary maintainability-tier issues. This division of labor instantiates the architectural rationale of Section 5.F: the Engine supplies a reproducible, CWE-grounded baseline, while findings that are security-relevant yet fall outside this taxonomy are delegated to the fine-tuned component. The first CWE-mapped weakness (CWE-327, at L3) produces an immediate drop to S_{sec}=1.25, consistent with the weakest-link formulation of Eq. 3. Across L3–L5, five of the nine CWE entries of Table 3 are empirically triggered: CWE-327 at L3; CWE-89 and CWE-798 at L4; and CWE-78 and CWE-95 at L5, with L5 alone triggering four distinct entries.

Read in the direction L5 → L1, the same ordering describes a sequence of successive corrections, each moving every score toward its maximum: the overall score rises monotonically from 2.48 (L5) to 9.28 (L1), and each individual security, complexity, and maintainability issue introduced at a given level is absent at the level below it. This provides preliminary evidence for the second clause of RQ5: a system that produces a strictly monotonic signal under a designed escalation of issues would, under the same scoring function, produce a strictly improving signal as those issues are corrected — the pattern required for ScoRev's scores to serve as a directional feedback signal in an iterative, AI-assisted refinement loop in which a model proposes edits and ScoRev's score change indicates whether each edit moved the code toward or away from the exemplary band of Table 2.

X. *Threats to Validity*

Detection Benchmark sample size (N=10). All ten files and their 29 annotations were designed and finalized before any system was run against them (Section 7.O), eliminating post-hoc selection as an explanation for the results of Sections 8.T and 8.U. The single sample on which the grounding ablation reverses (S6, Section 8.T) does not undermine the aggregate result, because the aggregate F_1 already reflects this reversal and the +0.190 gain persists despite it.

Ranking Benchmark sample size (N=5). With five ordered levels, $\rho = -1.000$ for the overall and maintainability scores is the most extreme value attainable and corresponds to an exact two-tailed p-value of $1/120 \approx 0.0083$ (Table 12), not the much smaller values that a large-sample approximation would (incorrectly) report at this N. The single inversion in the complexity series (L3, 7.84, slightly below L4, 8.14, a difference of 0.30) is the smallest possible deviation from perfect monotonicity at N=5 — a single adjacent-pair swap — and does not indicate a systematic ordering failure.

Silver-Standard category-level estimates (N=24 per category). The negative security correlation in the legacy/mixed category (Section 8.S) is a single category-level estimate at N=24, where individual-file scoring noise from any one of the three LLM judges can produce a sign reversal without altering the aggregate N=120 estimate (Table 7). This reversal does not undermine the aggregate security correlation, because the

aggregate already incorporates this category's contribution and remains positive.

Residual false positives. On S1, A reports two false positives describing a pair of conditional branches in `calculate_discounts` as redundant; the two branches in fact test distinct dictionary keys and are not redundant. On the clean files S5 and S10, A and B occasionally report stylistic suggestions on otherwise-correct code (for example, suggesting `re.findall` in place of `.split()`, or flagging a deliberate `ValueError` guard as a potential `ZeroDivisionError`). On S10, C additionally hallucinates a missing docstring on a file that contains one. These residual false positives are reflected in the FP counts of Table 9 and do not affect the relative comparisons of Sections 8.T and 8.U, which hold the matching protocol fixed across all four runs.

Detection Benchmark matching scope. Run A's F_1 of 0.630 measures the model's complementary output only and excludes issues already surfaced by the Engine in the findings block; when the Engine's direct detections are included, ScoRev's total issue coverage reaches 77% of Gemini 3 Flash-Lite's detections (Table 9, row E). The reported F_1 therefore represents a conservative lower bound on ScoRev's combined detection capability.

LLM-judge consensus as a validation reference. RQ1's second clause concerns the reliability of the validation reference itself. The inter-judge ceilings of Table 7 (0.394–0.647) indicate that the three-judge consensus is an imperfect reference independent of the Engine's behaviour: a documented leniency bias in LLM-based judging [47], [46] would depress Engine–consensus correlations regardless of how accurate the Engine's own scores are, and would do so in the direction observed (Section 8.S). This does not invalidate the Silver-Standard comparison as one input among several — the Detection Benchmark (Sections 8.T, 8.U) and Cross-Validation experiment (Section 8.V) provide convergent, non-LLM-judge-dependent evidence — but it bounds how much weight Section 8.S's absolute correlations should carry in isolation.

9. Future Work

There are several directions that can further enhance the capabilities of ScoRev. First, the framework can be extended to support additional programming languages beyond Python, making it applicable to a broader range of software projects and development environments. Such an extension would require language-specific static analysis components and quality metrics while preserving the framework's hybrid architecture.

Second, future versions of ScoRev can incorporate additional software quality dimensions, including reliability, performance efficiency, and scalability. Expanding the assessment beyond security, complexity, and maintainability would provide a more comprehensive evaluation of software quality and align the framework more closely with established software quality standards.

Another promising direction is the integration of Retrieval-Augmented Generation (RAG) techniques. By leveraging external knowledge sources, secure coding guidelines, and software engineering best practices, the semantic review component could generate more accurate, context-aware, and evidence-based recommendations.

As AI-generated software becomes increasingly prevalent, future research should investigate ScoRev's effectiveness in assessing and improving code produced by large language models. This would further establish the framework as a quality assurance layer for AI-assisted software development workflows.

In addition, a multi-agent architecture could be explored, where specialized agents independently analyze different quality dimensions and collaboratively produce a unified assessment. Such an approach may improve both coverage and review quality.

Future work may also examine the use of reinforcement learning to continuously improve the framework based on developer feedback and real-world review outcomes. This would enable ScoRev to adapt its recommendations and assessment strategies over time.

Finally, extending the analysis from individual source files to repository-level understanding would allow the framework to consider project architecture, dependencies, and cross-file interactions. Enhancing the explainability of AI-generated assessments would also improve transparency, user trust, and practical adoption in industrial software development environments.

10. Limitations

Despite its promising results, ScoRev has several limitations that should be acknowledged.

- **Python-Only Support:** The current implementation supports only Python source code. Although the overall architecture is largely language-independent, the static analysis engine, quality metrics, and rule sets were specifically designed for Python. Extending the framework to additional programming languages would require language-specific analyzers, metrics, and quality-assessment rules.
- **File-Level Analysis:** ScoRev evaluates software quality at the individual file level rather than the repository or system level. Consequently, architectural dependencies, inter-module interactions, cross-file data flows, and system-wide design concerns are not fully captured.
- **Dependence on Predefined Metrics and Calibration:** The deterministic scoring engine relies on predefined metrics, thresholds, and calibration parameters derived from software-engineering literature and documented modeling decisions. Alternative calibration strategies may produce different score distributions and quality assessments.
- **Knowledge-Distillation Constraints:** The semantic review component is trained through knowledge distillation and therefore inherits some limitations of its teacher model. Although grounding mechanisms reduce hallucinations and improve consistency, the model may still

generate incomplete, inaccurate, or context-dependent recommendations.

- **Limited Quality Dimensions:** The framework currently focuses on three software-quality dimensions: security, complexity, and maintainability. Other important attributes, such as reliability, performance efficiency, scalability, portability, and usability, are outside the scope of the current work.
- **Dataset Limitations:** The quality and diversity of the semantic review component depend on the underlying training dataset. Since the dataset was constructed primarily from open-source Python repositories, certain application domains and proprietary industrial software systems may be underrepresented.
- **Static Analysis Limitations:** Like all static-analysis approaches, the deterministic engine may still produce false positives and false negatives. Certain vulnerabilities, behavioral defects, and runtime-dependent issues require dynamic analysis or execution-based information that is unavailable through static inspection alone.
- **Resource and Validation Constraints:** This work was conducted within the time and computational limitations of an undergraduate graduation project. As a result, large-scale experimentation, extensive hyperparameter optimization, broader comparisons with state-of-the-art systems, and comprehensive industrial validation were beyond the scope of the project. Furthermore, the framework has not yet been evaluated through long-term industrial deployments or developer studies, leaving its impact on developer productivity, software quality improvement, and maintenance-cost reduction as important directions for future investigation.

11. Conclusion

In this paper we introduced ScoRev, a hybrid software quality assessment method that combines deterministic static analysis with a knowledge-distilled language model to provide explainable, reproducible, and actionable software-quality assessment. The framework provides a transparent multi-dimensional assessment methodology covering security, complexity, and maintainability through literature-based metrics, calibrated scoring functions, and line-anchored findings.

To support this approach, we built a large dataset from open-source Python repositories based on a deep knowledge-distillation pipeline with source code, static analysis output, quality scores, and semantic reviews. This dataset was then used to fine-tune a compact language model that can generate complementary issues, contextual insights, actionable recommendations, and independent quality score predictions but is still feasible for local deployment.

Demonstrations showed that deterministic grounding and task fine-tuning have a significant impact on system performance. Engine results improved issue detection, as well as distilled model results provided semantic observations and recommendations that are beyond rule-based reasoning alone. These findings suggest the need to combine deterministic software engineering and language model reasoning in a single assessment framework.

ScoRev provides a simple and cost-effective framework for automated code reviews and software quality assurance for AI-generated software and privacy-sensitive applications where transparency, reproducibility, and local deployment are important. These results highlight the potential of hybrid approaches to bridge the gap between metric-based quality assessment and semantic code comprehension and provide a more comprehensive and trustworthy software quality evaluation.

References

[1] A. Bessey *et al.*, "A few billion lines of code later: using static analysis to find bugs in the real world," *Communications of the ACM*, vol. 53, no. 2, pp. 66–75, 2010.

[2] S. McConnell, *Code Complete*, 2nd ed. Microsoft Press, 2004.

[3] B. Johnson *et al.*, "Why don't software developers use static analysis tools to find bugs?" in *ICSE*, 2013, pp. 672–681.

[4] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.

[5] B. Roziere *et al.*, "Code llama: Open foundation models for code," *arXiv:2308.12950*, 2023.

[6] H. Pearce *et al.*, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *IEEE S&P*, 2022, pp. 754–768.

[7] Y. Fu *et al.*, "Security weaknesses of copilot generated code in github," in *ICSE*, 2023.

[8] M. Beller *et al.*, "Analyzing the state of static analysis," in *SANER*, 2016, pp. 470–480.

[9] A. Fan *et al.*, "Large language models for software engineering: Survey and open problems," *arXiv:2310.03533*, 2023.

[10] S. Lu *et al.*, "A survey of large language models for code," *arXiv preprint arXiv:2311.07989*, 2023.

[11] D. Guo *et al.*, "Deepseek-coder: When the large language model meets programming," *arXiv:2401.14196*, 2024.

[12] Y. Liu *et al.*, "Your code secret can be easily stolen by llms," in *ICSE*, 2024.

[13] S. Ouyang *et al.*, "Llm is like a box of chocolates: the non-determinism of chatgpt in code generation," in *ASE*, 2023.

[14] ISO/IEC, "Iso/iec 25010: Systems and software quality models," ISO/IEC, Tech. Rep., 2023.

[15] V. Lenarduzzi, F. Pecorelli, N. Saarimaki, S. Lujan-Mora, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," *Journal of Systems and Software*, vol. 198, p. 111575, 2023.

[16] S. M. Abtahi and A. Azim, "Augmenting large language models with static code analysis for automated code quality improvements," *arXiv preprint arXiv:2506.10330*, 2025.

[17] V. Authors, "Enhancing static analysis with llms to detect software vulnerabilities," *IEEE LLM4Code Workshop*, 2025.

[18] Z. Li, M. Zhang, M. Wang *et al.*, "Codereviewer: Pre-training for automating code review activities," in *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 1548–1560.

[19] Anonymous, "Human-like code quality evaluation through llm-based recursive semantic comprehension," *arXiv preprint arXiv:2412.00314*, 2024.

[20] Z. Zheng *et al.*, "Race: Beyond correctness: Benchmarking multi-dimensional code generation for large language models," *arXiv preprint*, 2024.

[21] V. Raina, A. Liusie, and M. Gales, "Is llm-as-a-judge robust? investigating universal adversarial attacks on zero-shot llm assessment," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

[22] S. Ullah *et al.*, "Llms cannot reliably identify and reason about security vulnerabilities (yet?)," *arXiv preprint*, 2024.

[23] Z. Li, S. Dutta, and M. Naik, "Iris: Llm-assisted static analysis for detecting security vulnerabilities," *International Conference on Learning Representations (ICLR)*, 2025.

[24] I. Jaoua, O. B. Sghaier, and H. Sahraoui, "Combining large language models with static analyzers for code review generation," *arXiv preprint arXiv:2502.06633*, 2025.

[25] D. Geer *et al.*, "The myth of information security," *Queue*, vol. 2, no. 4, pp. 26–31, 2004.

[26] SonarSource, "Sonarqube documentation: Metrics and quality gates," 2024. [Online]. Available: https://docs.sonarsource.com

[27] FIRST, "Common vulnerability scoring system version 3.1 specification document," 2019. [Online]. Available: https://www.first.org/cvss/v3-1/specification-document

[28] University of Maryland Baltimore County, "Cwe to cvss mapping dataset," 2024.

[29] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[30] A. H. Watson and T. J. McCabe, "Structured testing: A testing methodology using the cyclomatic complexity metric," National Institute of Standards and Technology, Tech. Rep. NIST SP 500-235, 1996.

[31] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[32] D. Coleman, D. Ash, B. Lowther, and P. Oman, "Using metrics to evaluate software system maintainability," *Computer*, vol. 27, no. 8, pp. 44–49, 1994.

[33] S. Butler, M. Wermelinger, Y. Yu, and H. Sharp, "Relating identifier naming flaws and code quality," in *European Conference on Software Maintenance and Reengineering*, 2009, pp. 31–35.

[34] D. Lawrie, C. Morrell, H. Feild, and D. Binkley, "What's in a name? a study of identifiers," in *IEEE International Conference on Program Comprehension*, 2006, pp. 3–12.

[35] D. Goodger and G. van Rossum, "Pep 257 – docstring conventions," 2001. [Online]. Available: https://peps.python.org/pep-0257/

[36] G. van Rossum *et al.*, "Pep 484 – type hints," 2014. [Online]. Available: https://peps.python.org/pep-0484/

[37] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[38] C.-Y. Hsieh *et al.*, "Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes," *arXiv preprint arXiv:2305.02301*, 2023.

[39] M. Thelwall, "Can large language models be reliable evaluators?" *arXiv preprint*, 2023.

[40] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[41] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, 2023.

[42] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[43] Y. Moslem *et al.*, "Revisiting parameter-efficient fine-tuning for large language models," *arXiv preprint*, 2024.

[44] M. Shaw, "Writing good software engineering research papers," in *Proceedings of the 25th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2003, pp. 726–736.

[45] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[46] J. Gu *et al.*, "Judging the judges: A systematic study of position bias in LLM-as-a-judge," *arXiv preprint arXiv:2406.12624*, 2024.

[47] Anonymous, "LLM judge leniency bias relative to human expert panels," *arXiv preprint arXiv:2602.05110*, 2026,

[48] M. Borg *et al.*, "Ghost echoes: A large-scale study of maintainability subjectivity among professional developers," *arXiv preprint arXiv:2408.10754*, 2024,